

Проект: Побудова Мінімальної DBMS

1.1. Загальна Концепція

Протягом курсу ви розробляєте **один цілісний проект** — невелику одновузлову Систему Управління Базами Даних (СУБД).

Ключові компоненти:

- **Локальне Зберігання на Диску:** Власний або існуючий формат файлів, сторінок, записів.
- **Простий SQL-Фронтенд:** Інтерфейс командного рядка (CLI), парсер, Абстрактне Синтаксичне Дерево (AST).
- **Двигун Виконання Запитів:** Сканування таблиць (table scan), фільтри, проекція, сортування/обмеження (**ORDER BY / LIMIT**).
- **Просунуті Фічі (Обрані вами):** Одна або декілька додаткових функцій (наприклад, **JOIN**, агрегація, індекси, типи, мінімальна оптимізація).

1.2. Очікувані Результати (Target Expectations)

Проект повинен мати:

- Нормальну структуру коду.
- Автотести (мінімум для ключових аспектів: парсер, сховище, виконання).
- Логування / debug-режим.
- Коротку технічну документацію.

2. Фінальний Продукт (Deliverable)

Фінальна архітектура проекту матиме приблизно такий вигляд:

1. CLI / Frontend

- Запит через аргумент: **db-cli --query "SELECT name FROM users WHERE age > 21"**
- REPL-режим (опційно): **db-cli » db> ...**

2. Parser & AST

- Лексер та парсер для підмножини SQL (**SELECT**, **CREATE TABLE**, **INSERT**, пізніше **DELETE** / **CTAS** / **JOIN**).
- AST-структури з механізмом візуалізації/відладки (pretty-printer/debug-dump).

3. Catalog & Storage Engine

- **Каталог Схем:** Зберігання метаданих (таблиці, колонки, типи).
- **Формат Файлу Таблиці (приклад):** Сторінки, слотові сторінки, RID (record id).
- **Базові Операції:** Створити таблицю, вставити рядок, просканувати таблицю.

4. Query Execution Engine

- **Оператори:** **TableScan**, **Filter**, **Projection** (опційно **OrderBy**, **Limit**).
- **Інтерфейс Виконання:** Простий **Volcano-подібний** інтерфейс **next()** або батч-виконання (на ваш розсуд).
- **Планування:** Мапінг AST → (логічний) план → (фізичні) оператори.

5. Advanced features

Один або кілька з наступних пунктів:

- Один **JOIN**-тип (**hash join** / **nested loop**).
- Одна агрегація (**COUNT**, **SUM**, ...).
- Простий індекс (**in-memory B-tree** чи **heap + sorted index**).
- Мінімальний оптимізатор (pushdown фільтрів, проєкцій).
- Підтримка простого типу JSON/ARRAY.

3. Етапи Проєкту (Milestones)

Етап 1: Frontend & Parsing First

Створення CLI, парсера та AST. На цьому етапі система вже може парсити запити, валідувати їхню структуру та виводити AST або помилки.

Етап 2: Catalog & Storage

Реалізація зберігання даних, схем та примітивного стореджа.

Етап 3: Query Engine

Створення системи, яка виконує **SELECT**-запити над вашим сховищем.

Етап 4: Advanced Query Execution

Додавання елементів виконання, які становлять для вас найбільший інтерес.

Можливі напрямки (обрати 1–2):

- JOIN
- Aggregation
- Index
- Mini-optimizer
- Typesystem / JSON / Arrays
- Performance / Tooling
- Basic transactions
- Distributed execution